

see the effect of these support vector regression models also.

Let us now shift our focus to neural networks and TensorFlow. But we will definitely revisit other methods provided by scikit-learn in the next article in this series when we discuss unsupervised learning and clustering.

Neural networks and TensorFlow

The deployment of neural networks is the main strength of machine learning based applications. Let us go through a few more examples from the *nn* (primitive Neural Net Operations) module of TensorFlow to get better familiarity with the neural network jargon. We have already seen two activation functions called ReLU (rectified linear unit) and Leaky ReLU offered by the module *nn* of TensorFlow. There are other activation functions also offered by the module *nn*. The method *tf.nn.crelu()* computes the concatenated ReLU. The method *tf.nn.elu()* computes the exponential linear function. Soon, we will use one of these activation functions while training our first model using TensorFlow and Keras.

But before we move on to that section, let me share with you the details of an excellent visual tool that will help you understand the working of a neural network. This resource called 'Neural Network Playground' is provided by TensorFlow itself. You can visually add new neurons and new hidden layers to a neural network and then train that model. You can choose between activation functions like Tanh, Sigmoid, Linear and ReLU. Both classification and regression models can be analysed by using this tool. The effect of the training is shown as an animation. Figure 5 shows a sample neural network and the output provided by it. The tool is available online at the following URL, <https://playground.tensorflow.org>. This is a great resource for learning

about neural networks and this might be a good time in our journey through AI and machine learning to learn more about neural networks.

Our first trained model

Now, let us continue our discussion on the MNIST database of handwritten digits, a topic which we started to learn in the last article. In this article, we will train our model completely and test it with an image of a handwritten digit. The complete program, named *digit.py*, is relatively large. Hence, a part by part explanation is given here for better understanding. Line numbers are added to the program for ease of explanation.

```
1. import numpy as np
2. from tensorflow import keras, expand_dims
3. from tensorflow.keras import layers
4. num_classes = 10
5. input_shape = (28, 28, 1)
6. (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data( )
```

Lines 1 to 3 load the necessary packages and modules. Line 4 defines the number of classes as 10 because we are trying to classify the 10 digits (0 to 9). Line 5 defines the input shape as (28, 28, 1) because the data set we are going to use is a collection of 28 X 28 pixel grayscale images of handwritten digits. Line 6 loads this data set and divides it into training data and test data. More details about this data set were provided in the previous article in this series on AI and machine learning.

```
7. x_train = x_train.
   astype("float32") / 255
8. x_test = x_test.astype("float32") / 255
9. x_train = np.expand_dims(x_train, 3)
10. x_test = np.expand_dims(x_test, 3)
11. y_train = keras.utils.to_categorical(y_train, num_classes)
```

```
12. y_test = keras.utils.to_categorical(y_test, num_classes)
```

Lines 7 and 8 change the pixel values of the image, which is currently in the range [0, 255], to the new range [0, 1]. The method *astype()* is used for typecasting integer values to floating-point values. Lines 9 and 10 expand the shapes of arrays *x_test* and *x_train* (changing the shape from (60000, 28, 28) to (60000, 28, 28, 1)). The lists *y_train* and *y_test* contain an array of digit labels denoting the classes. In this case, it is 10 denoting the 10 digits from 0 to 9. Lines 11 and 12 convert the lists *y_train* and *y_test* to binary class matrices.

```
13. try:
14.     model = keras.models.load_model("existing_model")
15. except IOError:
16.     model = keras.Sequential(
        [
            keras.Input(shape=input_shape),
            layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
            layers.MaxPooling2D(pool_size=(2, 2)),
            layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
            layers.MaxPooling2D(pool_size=(2, 2)),
            layers.Flatten( ),
            layers.Dropout(0.5),
            layers.Dense(num_classes, activation="softmax"),
        ]
    )
17.     batch_size = 64
18.     epochs = 25
19.     model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
20.     model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, validation_split=0.1)
21.     model.save("existing_model")
```

Remember that model training is a processor intensive and memory consuming operation. Thus, with lines